

Universidad de Costa Rica

Facultad de Ingeniería

Escuela de Ingeniería Eléctrica

IE-0523 Circuitos Digitales II

I ciclo 2024

Reporte

Proyecto Final

Alonso José Jiménez Anchía B63561

Daniela Ulloa Barboza B77748

Paulette Pérez Monge B95916

Grupo 2

Profesor: Enrique Coen

6/7/2024

Índice

1. Resumen	1
2. Descripción Arquitectónica	2
2.1. Code-groups válidos empleados	2
2.2. Transmisor	3
2.3. Sincronizador	6
2.3.1. Diagrama de estados módulo de sincronización	6
2.3.2. Diagrama de bloques módulo del Sincronizador	7
2.4. Receptor	8
2.4.1. Diagrama de estados módulo de recepción	8
2.4.2. Diagrama de bloques módulo de recepción	9
2.5. PCS	10
3. Plan de Pruebas	11
3.1. Prueba 1: Operación correcta del módulo transmisor con su propio tester y testbench.	11
3.2. Prueba 2: Operación correcta del módulo sincronizador con su propio tester y testbench.	11
3.2.1. Prueba 2.1: Casos de error para el primer IDLE /I/	11
3.2.2. Prueba 2.2: Caso donde después de la primera comma se recibe dato válido pero no comma y luego la sincronización correcta	11
3.2.3. Prueba 2.3: Errores estando en sincronía	11
3.2.4. Prueba 2.4: Prueba diseñada para loopback	12
3.3. Prueba 3: Operación correcta del módulo receptor con su propio tester.	12
3.4. Prueba 4: PCS envía una trama completa de Ethernet configurado en modo de loopback.	13
4. Instrucciones de utilización de la simulación	14
4.1. Repositorio en GitHub	14
5. Ejemplos de los resultados	15
5.1. Prueba 1	15
5.2. Prueba 2.	15
5.3. Prueba 3.	16
5.4. Prueba 4.	18
6. Conclusiones y recomendaciones	19
6.1. Conclusiones	19
6.2. Recomendaciones	19

Índice de figuras

1.	Diagrama de bloques del transmisor.	3
2.	Máquina de estados de generación de code groups (transmisorCG) para transmisión.	3
3.	Máquina de estados de generación de ordered sets (transmisorOS) para transmisión.	4
4.	Máquina de estados para el módulo de Sincronización.	6
5.	Diagrama de bloques del sincronizador.	7
6.	Máquina de estados para el módulo de Recepción.	8
7.	Diagrama de bloques del receptor.	9
8.	Diagrama de bloques del PCS.	10
9.	Resultados de la prueba 1	15
10.	Resultados de la prueba 2.1	15
11.	Resultados de la prueba 2.2	16
12.	Resultados de la prueba 2.3	16
13.	Resultados de la prueba 2.4	16
14.	Resultados de la prueba 3.	16
15.	Resultados de la prueba 3.	17
16.	Resultados de la prueba 4.	18

Índice de tablas

1.	Code-groups de datos válidos implementados.	2
2.	Code-groups especiales válidos implementados.	2
3.	Sets ordenados definidos implementados generalmente.	2
4.	Salida según el Ordered set	5

1. Resumen

En el siguiente proyecto se diseña en SystemVerilog el bloque de la subcapa de codificación física (PCS, por sus siglas en inglés), en específico el bloque 1000BASE-X PCS [1] de acuerdo con las especificaciones de la cláusula 36 del estándar IEEE 802.3, con excepción de los diseños de los bloques Carrier Sense y Auto-negotiation. El diseño consiste de tres máquinas de estado: transmisor, sincronizador y receptor y la omisión de casos de error, omitiendo las capas inferiores. El transmisor se simplificó al máximo con respecto a la versión sugerida en la documentación, su correcto funcionamiento depende de suposiciones sobre las señales que envía GMII y posee un error en la señal de disparidad cuya causa es desconocida.

En el módulo de Sincronización se reciben los code groups provenientes del transmisor y son transmitidos hacia el receptor pero con la adición de un bit que determina si el ciclo del reloj donde fue enviado el dato fue par o impar, y además se encarga de generar una sincronización entre el transmisor y el receptor para que la comunicación entre ambos sea adecuada. Si no hay sincronía, este sub-bloque no podrá nunca enviar los datos del GMII de vuelta. Se logra implementar correctamente ante todas las posibilidades propuestas por el protocolo.

El módulo receptor recibe datos provenientes del módulo sincronizador, primeramente IDLES y cuando el sincronizador le avisa que hay sincronización de datos, entonces el receptor empieza a leer datos y a transmitírselos a la capa GMII. Aunque pueden existir errores desconocidos, el sistema completo mantiene la integridad de los datos de entrada y salida. El módulo receptor procesa adecuadamente las comas y SUDI, decodifica la disparidad, y envía valores correctos al GMII. El diseño del módulo PCS y su conexión con los sub-bloques se verificaron con formas de onda obtenidas, cumpliendo con las interfaces y máquinas de estado simplificadas del protocolo Ethernet, excluyendo casos de error y bloques específicos.

2. Descripción Arquitectónica

2.1. Code-groups válidos empleados

Para poder realizar las pruebas de los paquetes, se solicita escoger diez datos de la tabla 36: Valid data code-groups de la cláusula 36 del estándar IEEE 802.3. En la tabla 1 se muestran los datos (/D/) escogidos que serán transmitidos como parte del paquete. Además se agregan dos más ya que son necesarios para completar los IDLE.

Nombre Code-Group	Valor Hexadecimal	Valor en 8 bits HGF EDCBA	RD - (10 bits) abcdei fghj	RD + (10 bits) abcdei fghj
D27.0	1B	000 11011	110110 0100	001001 1011
D28.0	1C	000 11100	001110 1011	001110 0100
D29.0	1D	000 11101	101110 0100	010001 1011
D30.0	1E	000 11110	011110 0100	100001 1011
D31.0	1F	000 11111	101011 0100	010100 1011
D0.1	20	001 00000	100111 1001	011000 1001
D1.1	21	001 00001	011101 1001	100010 1001
D2.1	22	001 00010	101101 1001	010010 1001
D3.1	23	001 00011	110001 1001	110001 1001
D4.1	24	001 00100	110101 1001	001010 1001
D5.6	C5	110 00101	101001 0110	101001 0110
D16.2	50	010 10000	011011 0101	100100 0101

Tabla 1: Code-groups de datos válidos implementados.

También son necesarios ciertos code-groups especiales que serán parte de los sets ordenados (ordered sets) y estos se muestra en la siguiente tabla 2.

Nombre Code-Group	Valor Hexadecimal	Valor en 8 bits HGF EDCBA	RD - (10 bits) abcdei fghj	RD + (10 bits) abcdei fghj
K28.5	BC	101 11100	001111 1010	110000 0101
K23.7	F7	111 10111	111010 1000	000101 0111
K27.7	FB	111 11011	110110 1000	001001 0111
K29.7	FD	111 11101	101110 1000	010001 0111

Tabla 2: Code-groups especiales válidos implementados.

Por lo tanto, los sets ordenados definidos que contienen los code-groups especiales de la tabla anterior y que serán utilizados en la implementación del proyecto, se muestran en la tabla 3.

Código	Set Ordenado	Número de code-groups	Codificación
/I1/	IDLE 1	2	/K28.5/D5.6/
/I2/	IDLE 2	2	/K28.5/D16.2/
/R/	Carrier_Extend	1	/K23.7/
/S/	Start_of_Packet	1	/K27.7/
/T/	End_of_Packet	1	/K29.7/

Tabla 3: Sets ordenados definidos implementados generalmente.

2.2. Transmisor

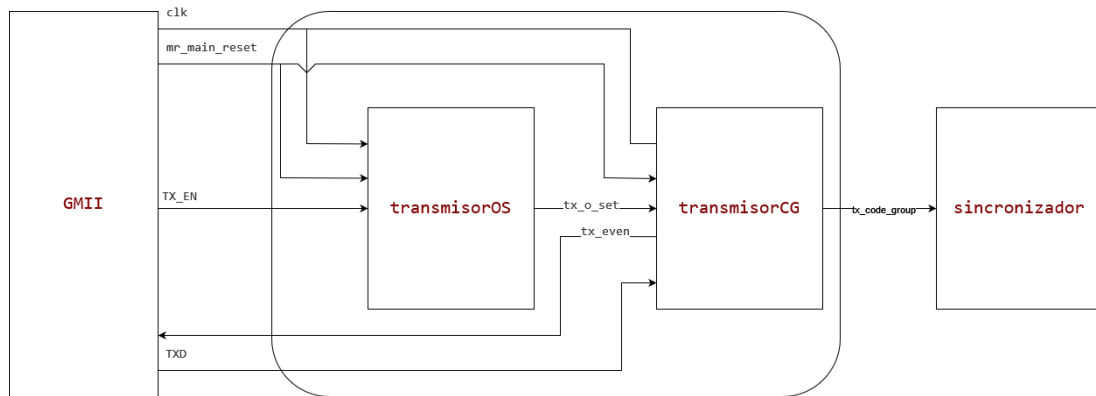


Figura 1: Diagrama de bloques del transmisor.

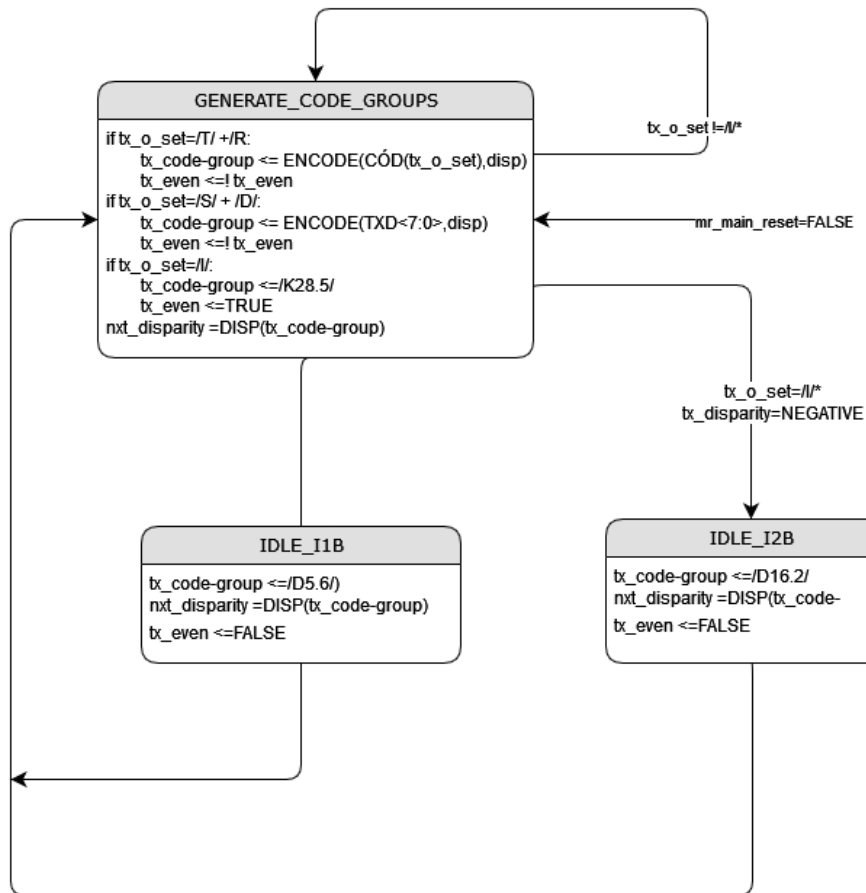


Figura 2: Máquina de estados de generación de code groups (transmisorCG) para transmisión.

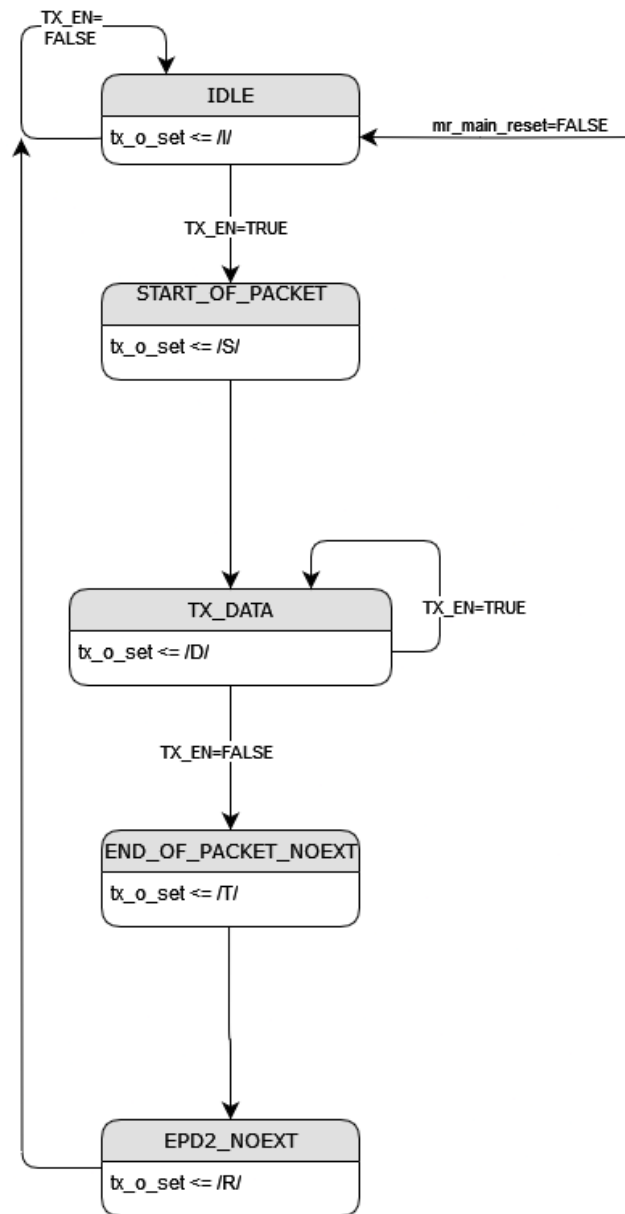


Figura 3: Máquina de estados de generación de ordered sets (transmisorOS) para transmisión.

Para el transmisor se hacen las siguientes suposiciones

- Aunque no se implemente en el código, porque no reflejaría nada, se supone que el GMII tiene una conexión con tx_even de forma que sólo va a mandar un TX_EN después de un ciclo con paridad negativa.
- El start_of_package se manda por el GMII al mismo tiempo que TX_EN se activa.

TransmisorCG

Según la figura 2, el transmisorCG tiene un estado llamado GENERATE_CODE_GROUPS, donde se decide qué sacar por la salida tx_code_group : si el tx_o_set enviado por transmisorOS es S o D, encodificamos el dato en la entrada TXD según la paridad con la función ENCODE, a partir de los grupos de la tabla 1. Si tx_o_set es cualquier otro valor, obtenemos su tx_code_group respectivo para el tx_o_set, según la tabla 4, lo que se representa de forma figurativa con el método CÓD(). Dependiendo si estamos en disparidad positiva (RD+) o negativa (RD-)

IDLE_I1B y IDLE_I2B permiten mandar el dato /D5.6/ o /D16.2/, respectivamente, mientras estemos en un IDLE y cambian la paridad.

En cada estado se calcula la disparidad del siguiente valor con la función **DIS()**. Si los últimos 4 bits del tx_code_group calculado tienen más 1s que 0s la disparidad es positiva y si es al contrario la disparidad es negativa. Si hay la misma cantidad analizamos los primeros 6 bits con el mismo criterio y si estos 6 bits también tienen la misma cantidad de 0s y 1s mantenemos la disparidad anterior.

Al resetear la máquina, el valor actual de tx_disparity se vuelve 0, de forma que el primer tx_code_group es un /K28.5/ en su codificación RD-. Éste está diseñado para que el siguiente tx_disparity sea un RD+. Sin embargo, como el cabio de estado utiliza la disparidad actual, eso significa que el estado en el siguiente ciclo de reloj será IDLE_I2B, mandando /D16.2/ al sincronizador en su codificación RD+, y como éste también está diseñado para que la siguiente disparidad sea negativa, alternaremos entre estos dos valores hasta que TX_EN se encienda. Por lo tanto, al inicio no accederíamos al estado IDLE_I1B.

Sus estados contaron con la siguiente codificación one-hot:

- GENERATE_CODE_GROUPS = 3'b001
- IDLE_I1B = 3'b010
- IDLE_I2B = 3'b100

TransmisorOS

Según la figura 3, el transmisorOS empieza mandando tx_o_set = I, el cuál es un parámetro, a transmisorCG, indicando que estamos en IDLE. Si TX_EN se pone en alto, vamos al estado START_OF_PACKET y ponemos tx_o_set en S. Un ciclo de reloj después llegamos a TX_DATA y mandamos tx_o_set = D. Si TX_EN se pone en bajo, vamos al estado END_OF_PACKET para mandar un tx_o_set = T, al siguiente ciclo vamos a EPD2_NOEXT que manda un tx_o_set = R y regresa a IDLE. Sus estados contaron con la siguiente codificación one-hot:

- IDLE = 5'b00001
- START_OF_PACKET = 5'b00010
- END_OF_PACKET_NOEXT = 5'b00100
- EPD2_NOEXT = 5'b01000
- TX_DATA = 5'b10000

El dato que transmisorCG debería mandar de acuerdo a cada tx_o_set viene indicado en la tabla 4

Ordered set	tx_code_group
/I/	/K28.5/ seguido de /D16.2/ o /K28.5/ seguido de /D5.6/
/R/	/K23.7/
/S/	/K27.7/
/T/	/K29.7/
/D/	ENCODE(TXD)

Tabla 4: Salida según el Ordered set

Los ordered sets contaron con la siguiente codificación one-hot:

- R = 5'b00001
- S = 5'b00010
- T = 5'b00100
- D = 5'b01000
- I = 5'b10000

2.3. Sincronizador

2.3.1. Diagrama de estados módulo de sincronización

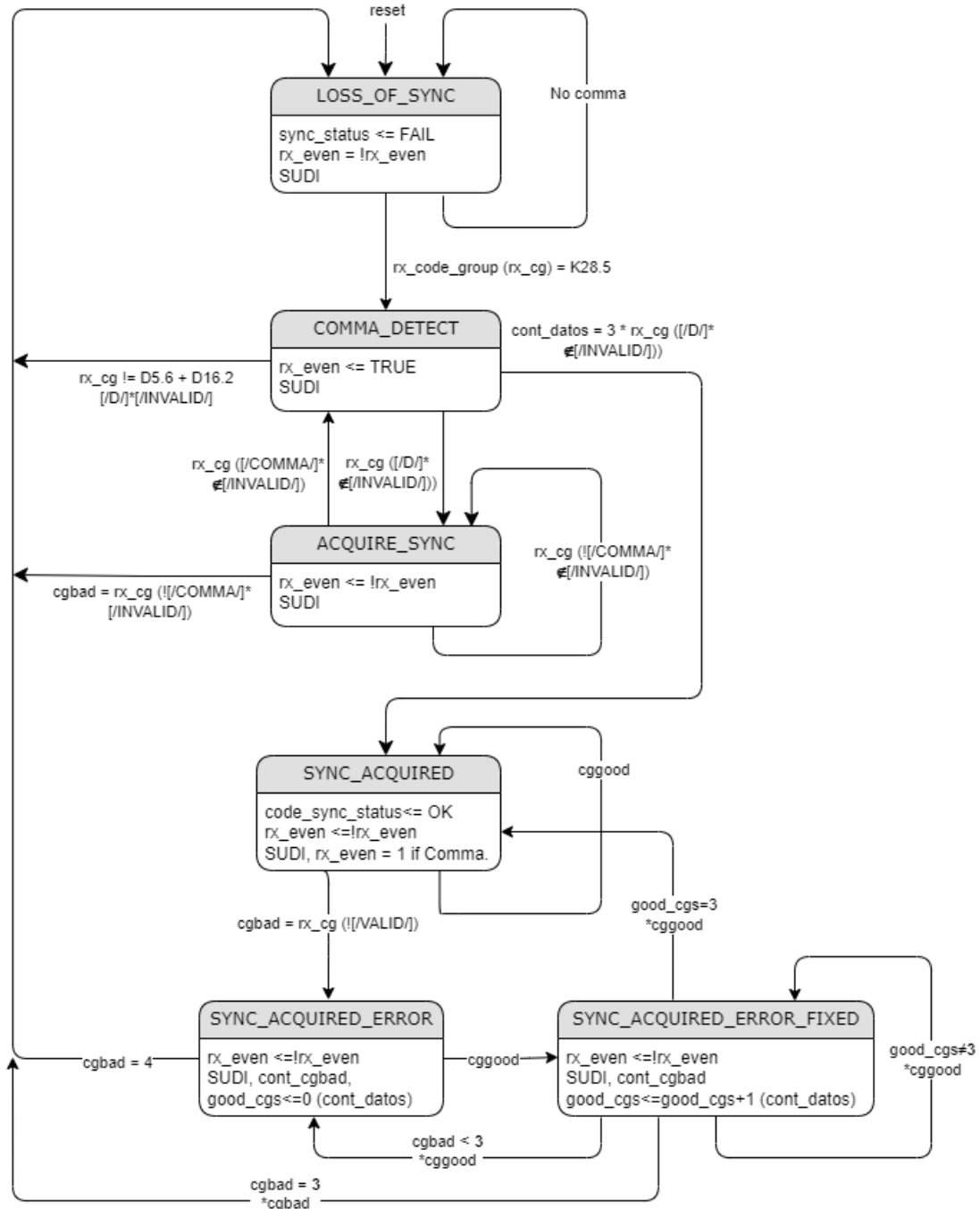


Figura 4: Máquina de estados para el módulo de Sincronización.

2.3.2. Diagrama de bloques módulo del Sincronizador

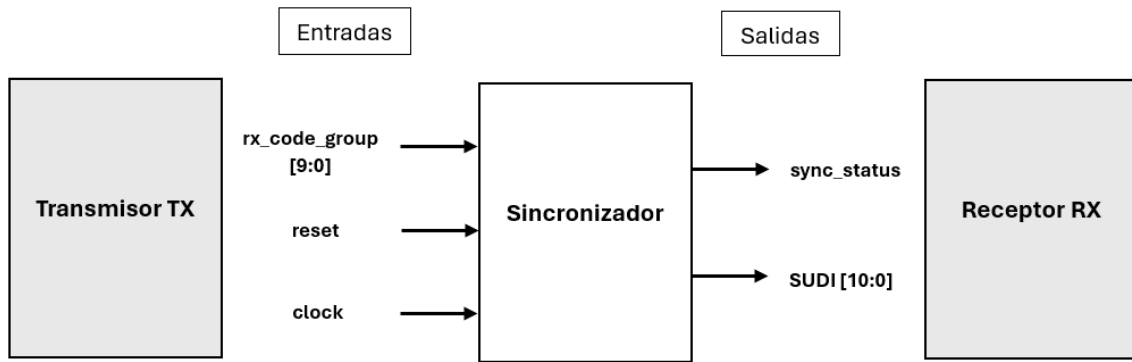


Figura 5: Diagrama de bloques del sincronizador.

Estados para el módulo de sincronización:

- **LOSS_OF_SYNC**: Es el estado donde se da la des-sincronización. Se espera la primera comma.
- **COMMA_DETECT**: A este estado se accede cuando se recibió una comma y se comprueba si el valor recibido es un dato especial parte de un IDLE. También se encarga de decidir cuando se accede al estado de sincronizado (**SYNC_ACQUIRED**).
- **ACQUIRE_SYNC**: A este estado se accede cuando se recibió una dato especial de tipo IDLE y se espera una comma.
- **SYNC_ACQUIRED**: En este estado permanece la máquina si hay una correcta sincronización y no ocurren errores de datos inválidos recibidos.
- **SYNC_ACQUIRED_ERROR**: En este estado se accede cuando existe la sincronización pero se detecta un error.
- **SYNC_ACQUIRED_ERROR_FIXED**: En este estado se permanece hasta que se reciban tres datos válidos luego de haber recibido uno inválido, para luego acceder nuevamente al estado de sincronización; en este estado no se pierde la sincronización.

Señales externas e internas de la máquina de sincronización:

- **clk**: Es una entrada y corresponde al reloj de la máquina.
- **reset**: Es una entrada que se encarga de resetear la máquina para generar valores conocidos en sus salidas y en registros internos.
- **rx_cg**: Señal de entrada de 10 bits que contiene los code groups provenientes del transmisor, son los datos generados en esta máquina en un formato de 10 bits en lugar de 8 y con dos posibles disparidades por dato. Es la misma señal que **tx_code_group** del transmisor.
- **rx_even**: Registro interno que define si el ciclo del reloj es par (**Even = 1**) o impar (**Odd = 0**).
- **estado**: Registro de 6 bits que define el estado actual.

- **cont_datos:** Este registro de 3 bits corresponde a un contador multifuncional a lo largo de la implementación del código, primeramente es utilizado para contar los datos especiales correspondientes recibidos correctamente luego de una comma o los IDLE completos, y luego cuando ya ocurre la sincronización, es utilizado para contar los datos válidos luego de haber recibido un error, para poder volver al estado de sincronización estable.
- **code_group:** Es un registro de 10 bits que almacena temporalmente el valor recibido de rx_cg para poder ser transmitido al receptor correctamente.
- **cg_valid:** Es una señal que determina si el dato recibido es válido o no, es el resultado de la función VALID.
- **cont_cgbad:** Este registro de 3 bits corresponde a un contador de datos inválidos recibidos luego de haber logrado la sincronización.
- **SUDI:** Es una salida de 11 bits generada por la máquina de sincronización la cual posee en el primer bit el valor de rx_even, y en los siguientes 10 bits el valor de rx_cg.
- **sync_status:** Señal que indica si se está sincronizado.

2.4. Receptor

2.4.1. Diagrama de estados módulo de recepción

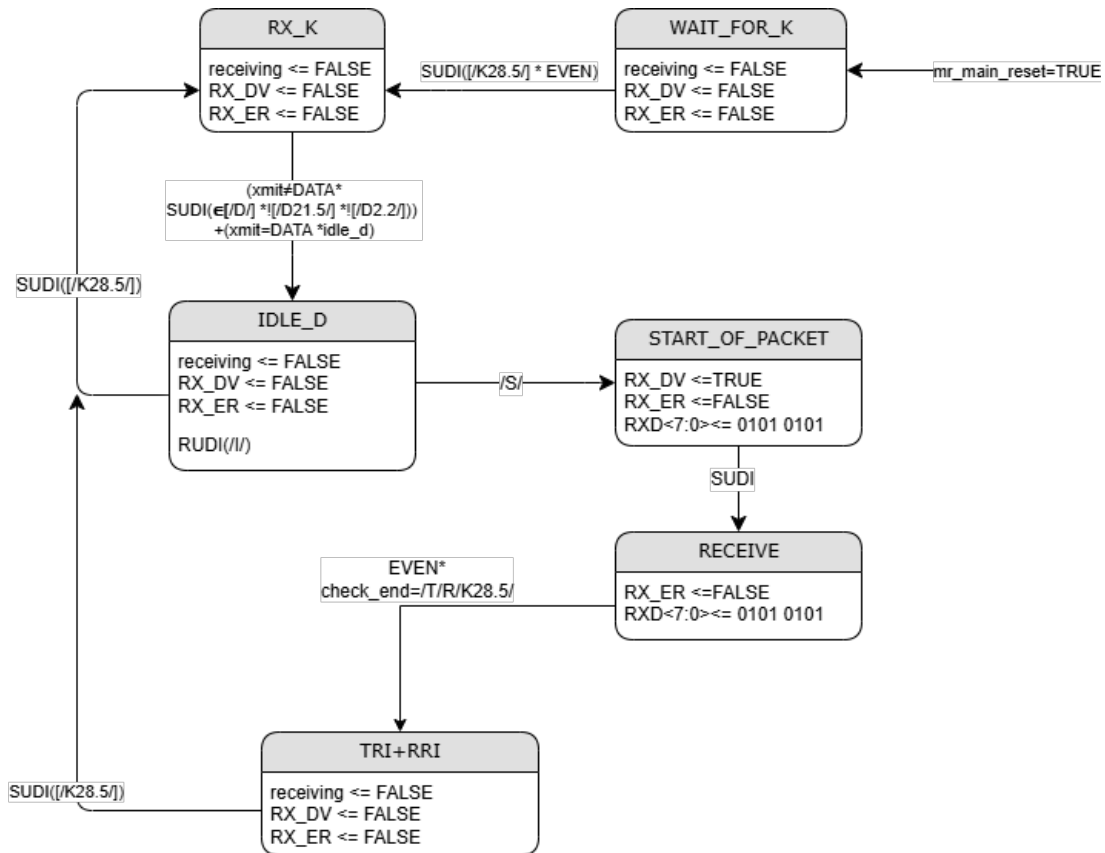


Figura 6: Máquina de estados para el módulo de Recepción.

2.4.2. Diagrama de bloques módulo de recepción

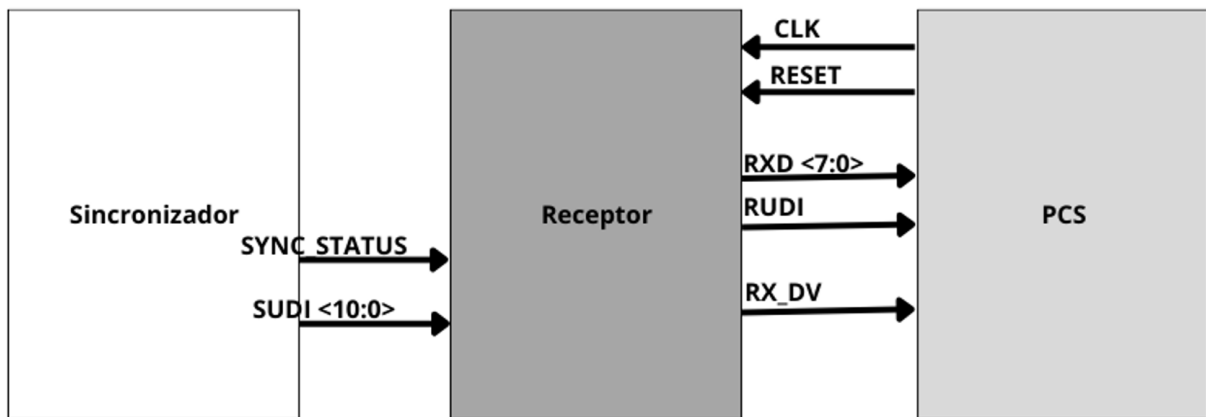


Figura 7: Diagrama de bloques del receptor.

La figura 6, muestra el diagrama de estados simplificado a partir del diagrama de estados para el bloque receptor del estándar de Ethernet según el bloque de PCS tipo 1000BASE-X. En la simplificación se unieron los estados RECEIVE y RX_DATA, en un solo estado llamado RECEIVE, ya que esto permite aplicar la función DECODE(/D/) a todos los datos /D/ que ingresen mientras se está en este estado, evitando problemas de lectura de los datos debido al CLK.

Estados para el módulo de recepción:

- WAIT_FOR_K: Estado inicial donde se espera la primer coma en un ciclo par ($/K28.5/*EVEN$) para pasar al siguiente estado. Solo se pasa por el una vez.
- RX_K: Se ha recibido una coma (K). Se espera /D/ para completar /I/ y así pasar al estado IDLE_D.
- IDLE_D: Se ha recibido un IDLE /I/, se cambia de estado cuando se recibe /S/.
- START_OF_PACKET: Se inicia análisis del paquete.
- RECEIVE: Se reciben datos y se decodifican para ser enviados al GMII.
- TRI_RRI: Terminando paquete de la manera correcta.

Señales para el módulo de recepción:

RX_DV: Señal que indica que el dato enviado es válido.

SUDI[10:0]: Esta entrada proviene del bloque Sincronizador y contiene en su bit más significativo la indicación de paridad y el resto de bits conforman los datos de entrada.

RUDI: Se pone en alto cada vez que se recibe un /I/

RXD[7:0]: Envía los /D/ decodificados al GMII.

Variables internas:

CHECK_END: Esta variable se pone en alto si detecta /T/R/R/ o /T/R/K28.5/, denotando la terminación normal del paquete (es decir, sin errores).

2.5. PCS

Diagrama de bloques módulo PCS

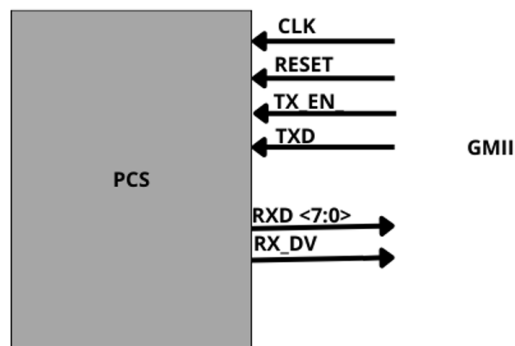


Figura 8: Diagrama de bloques del PCS.

Señales para el módulo de recepción:

CLK: Es una entrada que llega al receptor desde el PCS con una frecuencia determinada. El flanco activo de CLK es el flanco creciente.

RESET: Entrada de reinicio del receptor. Si RESET=1 el receptor funciona normalmente. En caso contrario, el receptor vuelve a su estado inicial y todas las salidas toman el valor de cero.

Las otras señales ya fueron explicadas en los módulos transmisor y receptor.

3. Plan de Pruebas

3.1. Prueba 1: Operación correcta del módulo transmisor con su propio tester y testbench.

Se mandan datos al azar desde el tester hasta que se manda un código FB representado el start of package, al mismo tiempo que se enciende TX_EN. Esto debe suceder después de un ciclo impar. Previamente se mandan /K28.5/ y /D16.2/ por el tx_code_group, de acuerdo con la disparidad elegida por default. Al encenderse TX_EN, los datos recibidos por TXD deben traducirse a un código de 10 bits de la tabla 1 según su disparidad, y calcular la siguiente según las reglas del estandar. Cuando TX_EN baje, el transmisor debe mandar un /K28.7/, un /K23.7/ y volver a mandar pares de códigos /I/ según la tabla 4.

3.2. Prueba 2: Operación correcta del módulo sincronizador con su propio tester y testbench.

Para poder comprobar el correcto funcionamiento del Sincronizador, con todas sus posibilidades de error y éxito consideradas en el diagrama de estados tanto del protocolo como el simplificado en esta implementación, fue necesario crear múltiples escenarios, los cuales serán descritos por medio de sub-pruebas, que son las siguientes:

3.2.1. Prueba 2.1: Casos de error para el primer IDLE /I/

Para poder completar esta prueba se debe enviar por medio de rx_cg un dato ya sea o válido o inválido especial o de cualquier tipo, siempre y cuando no corresponda a una comma, por lo tanto, que no sea ni K28.5 (+/-) o de disparidad positiva o negativa. Luego debe ser enviada una comma y un dato incorrecto; posteriormente puede ser un dato especial de IDLE pero que no sea una comma ya que al haber recibido una comma pero luego un dato no esperado, se reinicia el proceso y para volver a comenzar se debe recibir estrictamente una comma, y aunque un dato D5.6 o D16.2 (+/-) pertenecen a un IDLE /I/ no son commas, por lo que se permanece en el primer estado. Por último esta primera prueba se concluye ingresando un IDLE ya sea /I1/ (+/-) o /I2/ (+/-) y antes de recibir la segunda comma se ingresa otro dato no esperado. Al definir de esta manera se podrá observar que se respeta lo establecido en el diagrama de estados de la figura 4. Esta prueba se logra exitosamente y los resultados se muestran en la figura 10.

3.2.2. Prueba 2.2: Caso donde después de la primera comma se recibe dato válido pero no comma y luego la sincronización correcta

En esta prueba, se debe generar un IDLE /I/ como fue mencionado, pero luego se debe ingresar un dato especial de IDLE, en este caso se va a permanecer en el tercer estado ya que posee un loop para el caso de datos especiales tipo IDLE válidos pero no commas y por lo tanto, no se perderá el avance de sincronización, para salir de este estado se continúa con dos commas correctas más y se podrá observar que la señal sync_status se activa al recibir el tercer IDLE y por lo tanto se logra la sincronización correctamente. Esta prueba se logra exitosamente y los resultados se muestran en la figura 11.

3.2.3. Prueba 2.3: Errores estando en sincronía

Para esta prueba se experimentan todas las posibilidades del diagrama de estados, primero se ingresa un dato tipo start /S/ para simular que se comenzó el paquete, luego se envía un dato

inválido para acceder al quinto estado que es el de error, si se envía un dato válido se accede al último estado donde se comienza la cuenta de datos correctos, pero ahora se envía el máximo de errores permitidos para no perder sincronización que son 3 y se recibe uno válido, por lo que se cambia al último estado nuevamente donde se reciben 3 datos válidos y se recupera la sincronía estable. Luego se da otro caso donde se ingresan 3 datos inválidos, uno correcto pero otro inválido más, superando los 3 permitidos y por lo tanto, perdiendo total sincronización, esto significa que el proceso de sincronización vuelve a comenzar, solicitando 3 /I/ completos, logrando la sincronía y ahora 4 errores seguidos para perder sincronización directamente. Estas corresponden a todas las posibilidades de esta segunda etapa del sub-bloque de sincronización. Cada uno de estos escenarios son logrados correctamente, la simulación se torna muy extensa por lo que solo se adjunta un ejemplo de resultados de esta prueba en el presente documento, y se muestran en la figura 12.

3.2.4. Prueba 2.4: Prueba diseñada para loopback

Para poder completar esta prueba simplemente se encía tres /I/ seguidos con valores válidos y esperados, se agrega un /S/, luego todos los datos correctos deseados y luego se concluye el paquete con un /T/, un /R/ y por último otro /I/ y se deberá conseguir la sincronización y la correcta transmisión de los datos con los ciclos de reloj ya sea par e impar como corresponda, correctamente por medio del SUDI. Esta prueba se logra exitosamente y parte de los resultados se muestran en la figura 13. Para poder observar a más detalle los resultados se invita a ejecutar las simulaciones.

3.3. Prueba 3: Operación correcta del módulo receptor con su propio tester.

Cuando ocurre RESET se ingresa al estado WAIT_FOR_K en el siguiente flanco activo del CLK. Se recibe en SUDI un caracter K28.5 en un ciclo par y esto ocasiona que se pase al estado RX_K. Una vez en el estado RX_K se espera /D5.6/ ó /D16.2/, lo que culmina el recibimiento de un /I/. Esto último hace que se pase al estado IDLE_D; este estado indica que se ha recibido un /I/, por lo que activa la señal RUDI. Flujo del proceso se encicla entre los estados RX_K y IDLE_D, en espera del caracter /S/ y de la señal SYNC_STATUS en estado alto, esto debe ocurrir en estado IDLE_D para pasar al estado START_OF_PACKET. En START_OF_PACKET, simplemente se hace la transición al estado RECEIVE.

En el estado RECEIVE se decodifican todos los /D/ provenientes del SUDI y se reflejan los cambios en cada ciclo del CLK en la señal de salida RXD. El sistema se encicla en este estado RECEIVE hasta que se recibe al secuencia /T/,/R/ y /K28.5/*EVEN, la variable interna check_end se pone en estado alto justo cuando recibe esa secuencia, lo cual le indica al sistema que debe pasar al estado TRI_RRI. En estado TRI_RRI laS señales RX_DV y la señal interna receiving se pone en estado bajo indicando que el GMII no debe recibir más datos y que no se están recibiendo más /D/ válidos, respectivamente. Para salir del estado TRI_RRI se debe recibir una coma en estado par, esto hace que el siguiente estado sea RX_K y el sistema se vuelve a enciclar en los estados RX_K y IDLE_D porque es aquí donde el sistema recibe /I/. El diseño pasó la prueba.

3.4. Prueba 4: PCS envía una trama completa de Ethernet configurado en modo de loopback.

La prueba 4 cumple con todo lo estipulado en las pruebas 1, 2 y 3 descritas anteriormente, menos los casos donde hay error. Además, ahora sale a la vista las entradas y salidas del módulo PCS, cuyas entradas son las del transmisor y cuyas salidas son las del receptor. El objetivo de esta prueba es simplemente verificar que el correcto funcionamiento de los bloques anidados dentro del módulo PCS, puesto que las otras pruebas ya verificaron el correcto funcionamiento de estos ahora llamados subbloques.

4. Instrucciones de utilización de la simulación

4.1. Repositorio en GitHub

El código completo se encuentra en el repositorio del proyecto en el siguiente enlace: https://github.com/ecoenucr/1S2024_G2. Para ejecutar las pruebas, clone el código en su máquina linux. Después diríjase a la carpeta donde se encuentran los archivos y ejecute en su terminal el comando *make*, éste ejecutará las 4 pruebas en orden. Se abrirá una ventana de GTKWave con una prueba y después de cerrarla, la siguiente se abrirá automáticamente con la siguiente prueba.

5. Ejemplos de los resultados

5.1. Prueba 1

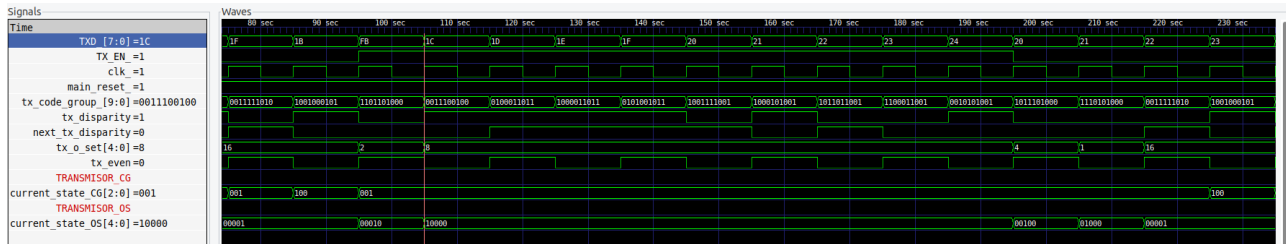


Figura 9: Resultados de la prueba 1

La figura 9 presenta una muestra de ejecución mayoritariamente correcta del código del transmisor. Al rededor del segundo 75, GMII manda datos, pero como TX_EN no está encendido, sólo se transmite /K28.5/ seguido de /D16.2/, siguiendo el comportamiento esperado si empezamos con disparidad negativa. Posteriormente TX_EN se activa en el momento adecuado, tx_o_set es un S y se deja pasar el caracter correspondiente. En el ciclo de reloj siguiente empezamos a mandar otros datos, traducidos correctamente con ENCODE. Cuando TX_EN se apaga, se mandan los caracter de los tx_o_set /T/ y /R/ y volvemos a mandar pares de /I/. La señal tx_even alterna como fue estipulado en el código.

Estos resultados tienen el problema de que, cuando tx_o_set=/D/, el valor de nxt_tx_disparity no pasa a tx_disparity, siendo estas las dos partes de un flip-flop. La primera variable calcula la disparidad correctamente, según las función DIS(). Se discutió con el profesor sobre la causa de éste problema y no se pudo dar con la solución, por lo que, en este respecto, la máquina no funciona como debe, **ni calcula el código adecuado para la disparidad**. Sin embargo, como la disparidad es una variable interna del transmisor e igualmente se envía un código correspondiente a TXD, la traducción hecha por el transmisor será la misma que si funcionara correctamente. Es preciso reiterar que nxt_tx_disparity sí obtiene el valor correcto, de acuerdo al estandar, y que ENCODE utiliza la disparidad que recibe para calcular la salida correspondiente, aunque sea la disparidad incorrecta. Gracias a esto no se puede hacer una prueba que utilice el estado IDLE_I1B, pues no se puede controlar el valor de tx_disparity.

5.2. Prueba 2.

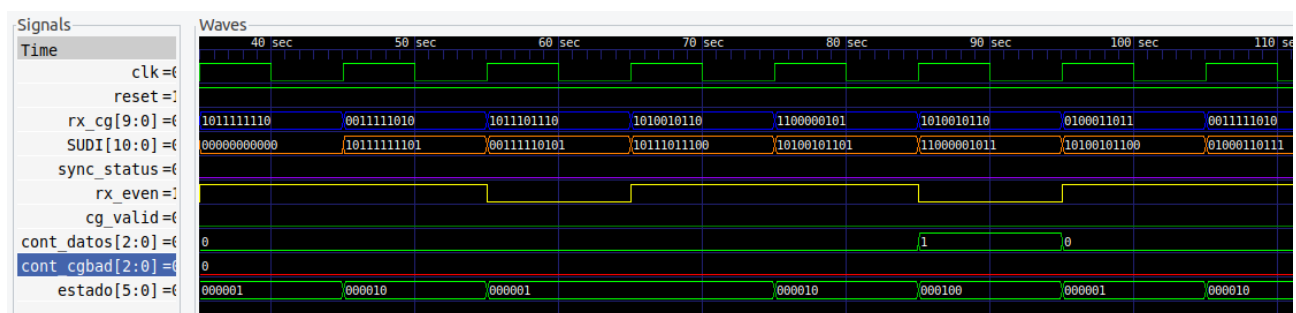


Figura 10: Resultados de la prueba 2.1

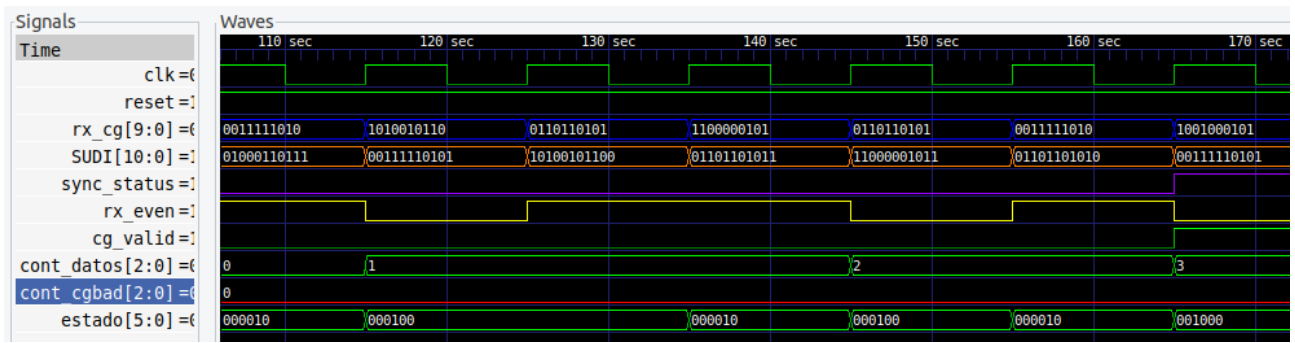


Figura 11: Resultados de la prueba 2.2

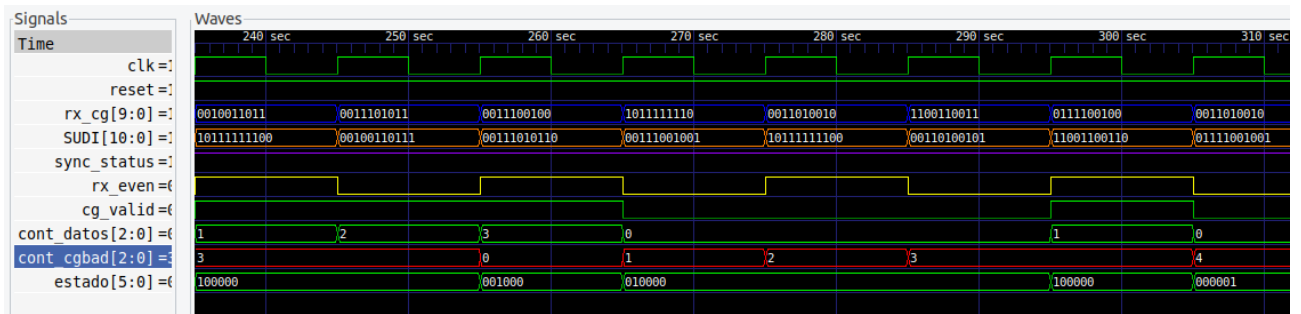


Figura 12: Resultados de la prueba 2.3

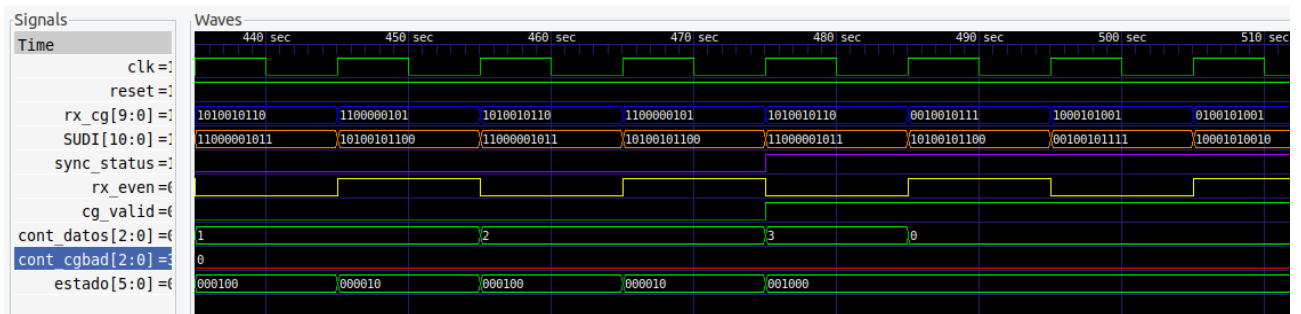


Figura 13: Resultados de la prueba 2.4

5.3. Prueba 3.

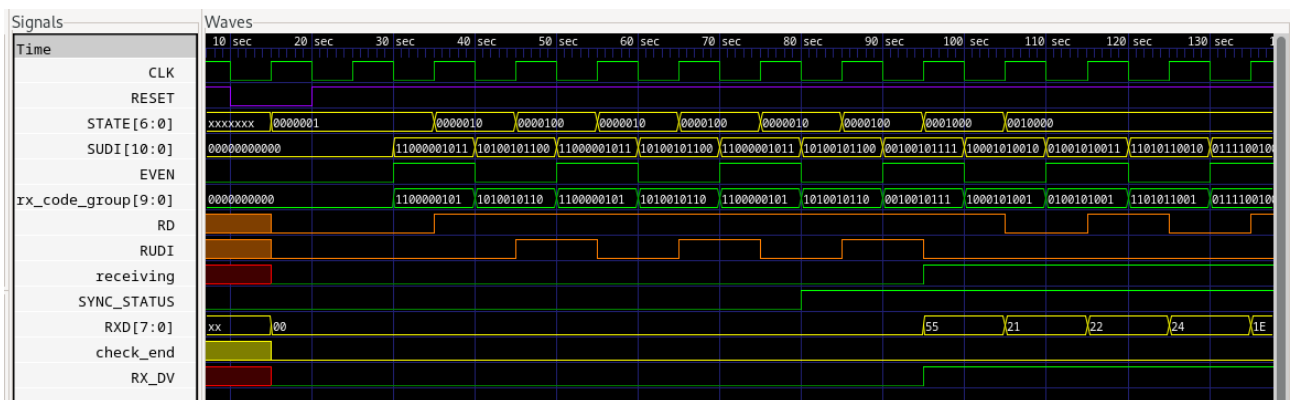


Figura 14: Resultados de la prueba 3.

En la figura 14, se aprecia como la primera parte de la prueba 3 pasa la prueba según el diseño implementando. Primeramente, note que apenas se activa el RESET, en el siguiente flanco positivo del reloj se pasa al estado WAIT_FOR_K cuyo valor en ONE HOT es 000001, valor que se presenta en la señal STATE. Consecutivamente, la simulación muestra como estando en el estado WAIT_FOR_K, llega una coma (carácter K28.5) en la señal SUDI (ver tabla 2). Note a su vez que la señal EVEN está en estado alto, lo que representa que esa coma se recibió en un ciclo par, lo que indica que habrá cambio de estado. En efecto, al segundo 35 se da el cambio de estado al estado RX_K. Luego, se observa como se pasa al estado IDLE_D después de recibir un D5.6 con disparidad positiva (ver tabla 2). También note que este tipo de datos especiales para los IDLE se reciben en un ciclo impar, verificado por la señal EVEN en estado bajo y el valor de 0 en SUDI[0]. Luego de recibir este dato, se ha recibido un /I/ completo, por lo que la señal RUDI se pone en alto. Posteriormente, note como el sistema se encicla en los estados RX_K y IDLE_D por 4 ciclos de reloj más, esto es claro en la señal STATE donde solo se encuentran los valores ONE HOT de 000010 y 000100.

Luego, se recibe el carácter /S/ alrededor del segundo 95 ya con la señal SYNC_STATUS proveniente del sincronizador en estado alto, lo que permite el paso al estado START_OF_PACKET. Observe como un ciclo después se pasa al estado RECEIVE, y justo al entrar en este estado se pone en alto la señal RX_DV y note también que la señal de salida RXD coloca el valor de 55 codificado en hexadecimal de acuerdo al protocolo para avisarle a GMII que se van a enviar datos válidos. Note que estando en el estado RECEIVE, se empiezan a enviar a través del RXD los códigos recibidos del sincronizador, los cuales son decodificados debido a que la señal de entrada SYNC_STATUS se puso en alto un tiempo atrás.

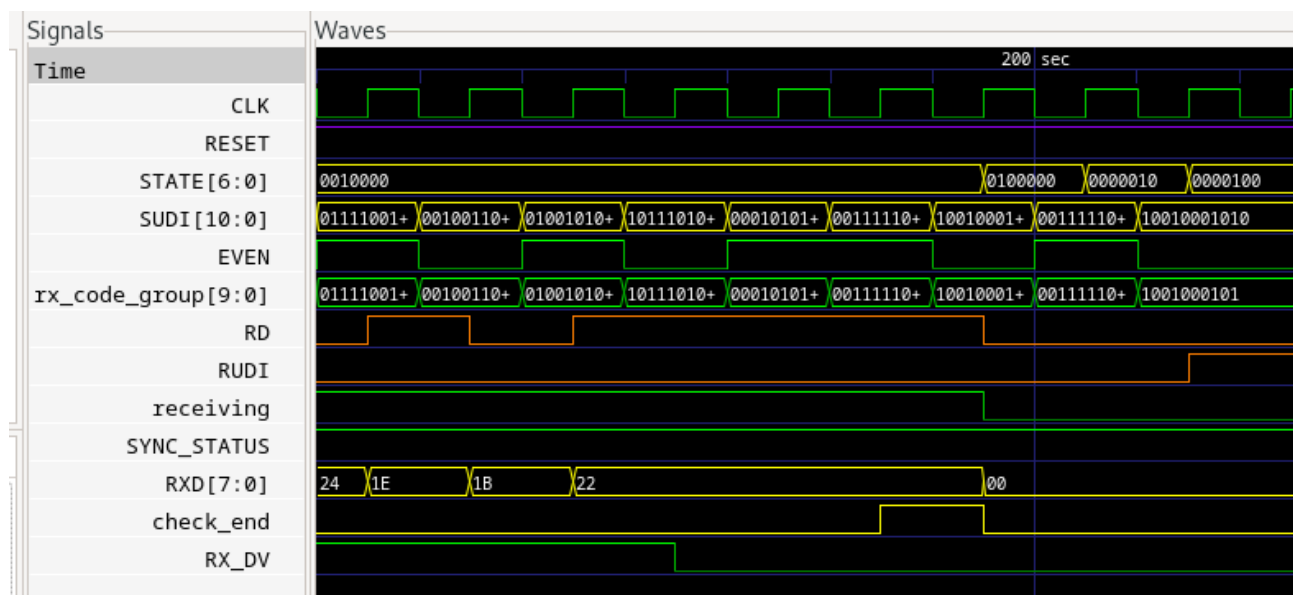


Figura 15: Resultados de la prueba 3.

En la figura 15, se aprecia la llegada de la secuencia /T/, /R/ y /K28.5/*EVEN, con la cual se activa la señal CHECK_END, la cual permite el pase al estado final TRI_RRI. Después de que RXD envió el valor 22 en hexadecimal, 3 ciclos de CLK después se activa la señal CHECK_END, la se activa justo después de recibir esos 3 caracteres seguidos. Note que 1 ciclo de reloj después de RXD envió un 22 a GMII, la señal RX_DV se puso en estado bajo, para que GMII no reciba valores repetidos. Ya en el estado TRI_RRI, la salida RXD toma su valor por default que es 0. Luego, se recibe nuevamente un carácter de /K28.5/*EVEN con el cual se pasa al estado RX_K, justamente porque se ha recibido una coma, y nuevamente el sistema termina por enciclarse en los estados RX_K y IDLE_D, esperando nuevamente el carácter /S/.

5.4. Prueba 4.

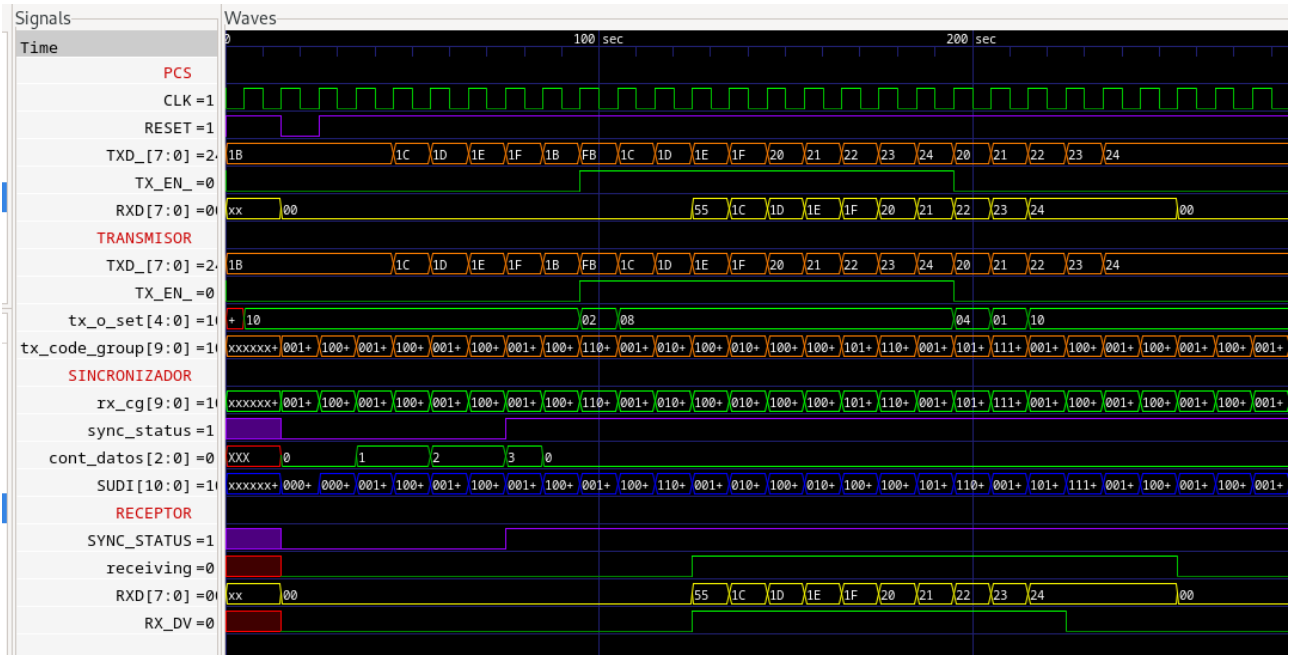


Figura 16: Resultados de la prueba 4.

En la figura 16, se observa el correcto funcionamiento de la prueba 4, es decir, todos los datos recibidos desde el GMII por el transmisor, son codificados y decodificados adecuadamente, resultando en los mismos valores transmitidos por el receptor al GMII algunos ciclos después debido a la latencia del sistema. Las pruebas 1, 2 y 3 ya cubren con exhaustividad el correcto funcionamiento de los sub-bloques, aquí lo importante son las señales del módulo PCS, que simplemente son las entradas del transmisor y las salidas del receptor, demostrando que operan todos los bloques correctamente, por lo que se concluye que hay una correcta conexión entre los mismos. Luego, note que los datos /D/ que se envían desde el transmisor cuando se da el TX_EN son los mismos que transmite el receptor en su señal RXD, y que la señal RX_DV se baja en un tiempo adecuado para evitar el envío de datos repetidos.

6. Conclusiones y recomendaciones

6.1. Conclusiones

- Aún si se piensa que el código está bien, puede haber errores de fuente desconocida, como fue el caso del transmisor. A pesar de esto, el resto de su funcionamiento fue el adecuado.
- Aunque la disparidad no sea la correcta, si el receptor y el sincronizador están adecuadamente implementados, los datos de entrada del sistema completo serán los mismos que los de salida.
- Se logra el diseño de un módulo receptor adecuado que opera de forma correcta con los módulos de sincronización y transmisor, de acuerdo al recibimiento de los datos respectivos en el tiempo adecuado.
- Se logra desarrollar adecuadamente el módulo de sincronización considerando todos los posibles casos de error aunque no fue solicitado, es de suma importancia para la comprensión correcta de este sub-bloque del PCS.
- Asimismo, se logra que el módulo receptor reciba las comas en el ciclo adecuado, que procese el SUDI correctamente, que active las señales adecuadas en el estado adecuado, que decodifique según la disparidad recibida y que envíe al GMII los valores correctos, todo esto de acuerdo al protocolo de Ethernet.
- Se logra el diseño del módulo PCS y una conexión exitosa, verificada por las formas de onda obtenidas, entre los módulos transmisor, sincronizador y receptor, convirtiéndose estos en sub-bloques anidados al módulo PCS cumpliendo con lo que establece el protocolo de Ethernet.
- El módulo PCS cumple con las interfaces (entradas y salidas) y con las máquinas de estado de acuerdo al protocolo de Ethernet, eso sí, simplificado y no considerando casos de error, ni los bloques CARRIER-SENSE ni AUTO-NEGOTIATION.

6.2. Recomendaciones

- Leer la documentación de Verilog para enterarse de fuentes de error evitables.
- Mejorar la puesta en bajo de la señal SYNC.STATUS luego de terminar el envío de una trama de datos, esto porque el receptor podría recibir señales inadvertidas.
- Realizar pruebas con cada avance pequeño que se realice, no esperar hasta el final del diseño para probar el módulo ya que será más difícil encontrar los fallos en la simulación. Cuando cada módulo esté funcionando correctamente hacer uniones individuales: Transmisor - Sincronizador, y Sincronizador - Receptor, y así la unión de los tres será más sencilla y existirá menor posibilidad de generación de errores.
- Errores en algunas señales pasaron inadvertidas como lo fue la puesta en bajo señal de TX_EN en un momento inadecuado, dejando pasar valores al recepto y sincronizador que no debían pasarse e identificadda por una persona externa al trabajo. Por lo que se sugiere analizar las señales con minuciosidad.

Referencias

- [1] “Ieee standard for ethernet,” *IEEE Std 802.3-2022 (Revision of IEEE Std 802.3-2018)*, pp. 1–7025, 2022.